

Docente: Jorge Izaguirre

Materia: Instalación, Mantenimiento y Reparación de Redes Informáticas

Nivel: 7mo Año - Técnico en Informática Personal y Profesional

Material Didáctico: Protocolos Fundamentales de Red y Diferencias entre TCP y UDP

Los temas a cubrir son la revisión de HTTP, FTP, TELNET, SMTP, POP3 y UDP, explicando su función y campo de aplicación dentro de la suite TCP/IP, y las diferencias entre TCP y UDP (orientado a conexión vs. no orientado a conexión).

Introducción a los Protocolos de Red

En el vasto universo de las redes informáticas, la comunicación fluida y eficiente es la piedra angular. Para que dos dispositivos puedan "hablar" entre sí, necesitan un conjunto de reglas comunes que dicten cómo se envían, reciben e interpretan los datos. Estas reglas son lo que se conoce como **protocolos de red**.

Imagine que desea enviar una carta a un amigo. Para que la carta llegue a su destino y su amigo pueda entenderla, se necesita un acuerdo sobre varios aspectos: el idioma en que se escribe, el formato de la dirección, la forma de pegar el sello, y cómo se entregará (por correo, mensajero, etc.). En las redes, los protocolos cumplen un papel similar, estableciendo el "idioma" y las "reglas de envío" para los datos.

Dentro de la arquitectura de red más utilizada, la suite de protocolos TCP/IP, existen diversos protocolos, cada uno con una función específica. Se explorarán algunos de los más importantes, analizando su propósito, su funcionamiento y sus aplicaciones.

Revisión de Protocolos de Aplicación Clave

Los protocolos de aplicación se encuentran en la capa superior del modelo TCP/IP (la capa de aplicación) y son los que interactúan directamente con las aplicaciones de usuario. Permiten que programas como navegadores web, clientes de correo electrónico o programas de transferencia de archivos realicen sus tareas.

1. HTTP (HyperText Transfer Protocol)

- ¿Qué es HTTP?
HTTP es el protocolo fundamental para la World Wide Web. Es el lenguaje que utilizan los navegadores web para comunicarse con los servidores web. Cuando se escribe una dirección web (URL) en el navegador, se está solicitando un documento (una página web, una imagen, un video) al servidor web, y HTTP es el protocolo que facilita esta solicitud y la entrega de la respuesta.
 - ¿Por qué se necesita HTTP?
Se necesita HTTP porque la web es un sistema distribuido de información. Los documentos y recursos están alojados en diferentes servidores en todo el mundo. HTTP proporciona un mecanismo estandarizado para que los clientes (navegadores) soliciten estos recursos a los servidores y para que los servidores los entreguen. Sin HTTP, cada sitio web o servidor podría usar su propio método de comunicación, lo que haría imposible la interoperabilidad y el acceso universal a la información.
 - ¿Cómo funciona HTTP?
HTTP opera sobre un modelo de solicitud-respuesta.

1. **El cliente (navegador) inicia una conexión TCP** con el servidor web en el puerto 80 (para HTTP estándar) o 443 (para HTTPS, que es HTTP sobre SSL/TLS).
 2. **El cliente envía una solicitud HTTP** al servidor. Esta solicitud incluye:
 - Un **método HTTP**: Indica la acción que el cliente quiere realizar (ej. GET para solicitar un recurso, POST para enviar datos, PUT para subir un recurso, DELETE para eliminar un recurso).
 - La **URL** del recurso que se solicita.
 - La **versión de HTTP** que el cliente está utilizando.
 - **Encabezados HTTP**: Contienen información adicional, como el tipo de navegador, el idioma preferido, o cookies.
 - Opcionalmente, un **cuerpo del mensaje**: Utilizado principalmente con métodos como POST para enviar datos al servidor (ej. datos de un formulario web).
 3. **El servidor procesa la solicitud** y genera una respuesta.
 4. **El servidor envía una respuesta HTTP** al cliente. Esta respuesta incluye:
 - Una **línea de estado**: Indica la versión de HTTP, un código de estado numérico (ej. 200 OK, 404 Not Found) y una descripción textual del estado.
 - **Encabezados de respuesta**: Contienen información sobre el servidor, el tipo de contenido que se envía, la fecha, etc.
 - Un **cuerpo del mensaje**: Contiene el recurso solicitado (ej. el código HTML de una página web, una imagen, etc.).
 5. **El cliente recibe la respuesta** y renderiza el contenido (ej. muestra la página web).
- **Ejemplo de aplicación**: Cuando se ingresa www.google.com en el navegador, este envía una solicitud HTTP GET al servidor de Google. El servidor responde con el código HTML de la página principal de Google, que el navegador luego interpreta y muestra. Otro ejemplo es cuando se envía un formulario en una página web; el navegador típicamente usa un método HTTP POST para enviar los datos del formulario al servidor.

2. FTP (File Transfer Protocol)

- ¿Qué es FTP?

FTP es un protocolo estándar de red utilizado para la transferencia de archivos entre un cliente y un servidor en una red de computadoras. Fue uno de los primeros protocolos desarrollados y sigue siendo relevante para ciertas tareas de transferencia de archivos.

- ¿Por qué se necesita FTP?

Se necesita FTP para transferir archivos de manera eficiente y confiable entre sistemas remotos. Aunque existen alternativas más modernas (como SFTP o SCP), FTP sigue siendo útil para la transferencia masiva de archivos o para sistemas heredados. Proporciona un método estandarizado para subir y descargar archivos, gestionar directorios remotos y autenticarse para el acceso a archivos.

- ¿Cómo funciona FTP?

FTP es único porque utiliza dos canales de comunicación TCP separados para una sola conexión:

- **Canal de control (Puerto 21)**: Utilizado para el intercambio de comandos y respuestas entre el cliente y el servidor. Aquí se negocian las operaciones (listado

de directorios, cambio de permisos, inicio de transferencia) y se maneja la autenticación.

- **Canal de datos** (Puerto 20 o un puerto dinámico): Utilizado para la transferencia real de los datos del archivo.
El FTP puede operar en dos modos principales para la transferencia de datos:
- **Modo Activo:** El cliente abre un puerto arbitrario para la conexión de datos y le informa al servidor la dirección IP y el número de puerto para que el servidor inicie la conexión de datos. Esto puede ser problemático con firewalls del lado del cliente.
- **Modo Pasivo:** El cliente envía un comando PASV al servidor, y el servidor responde con una dirección IP y un número de puerto en el que el servidor "escuchará" para una conexión de datos entrante desde el cliente. El cliente entonces inicia la conexión de datos al puerto especificado por el servidor. Este modo es más amigable con los firewalls.
- **Ejemplo de aplicación:** Un desarrollador web utiliza FTP para subir los archivos de su sitio web desde su computadora local a un servidor web remoto. También se puede usar para descargar software o actualizaciones desde un servidor FTP público.

3. TELNET (Teletype Network)

- ¿Qué es TELNET?

TELNET es un protocolo de red que permite a un usuario conectarse a una computadora remota (servidor) y ejecutar comandos en ella como si estuviera sentado físicamente frente a la terminal de esa máquina. Proporciona una interfaz de línea de comandos.

- ¿Por qué se necesita TELNET?
Históricamente, TELNET fue esencial para la administración remota de sistemas y para acceder a servicios de red basados en texto. Permitió a los administradores gestionar servidores desde cualquier ubicación sin necesidad de una interfaz gráfica. Hoy en día, su uso es limitado debido a importantes deficiencias de seguridad.
- ¿Cómo funciona TELNET?
TELNET opera sobre TCP, típicamente usando el puerto 23.
 1. El cliente TELNET establece una conexión TCP con el servidor TELNET.
 2. Una vez establecida la conexión, cualquier carácter que el usuario escribe en el cliente es enviado al servidor.
 3. El servidor procesa el comando y envía la respuesta de vuelta al cliente, que la muestra en la terminal.La comunicación entre el cliente y el servidor se realiza en texto plano, lo que significa que la información (incluyendo nombres de usuario y contraseñas) no está cifrada y puede ser interceptada fácilmente.
- **Ejemplo de aplicación:** Antes de SSH, un administrador de sistemas podía usar TELNET para acceder a un servidor Linux remoto y ejecutar comandos como `ls` para listar archivos o `reboot` para reiniciar el servidor. Sin embargo, debido a su inseguridad, TELNET ha sido ampliamente reemplazado por SSH (Secure Shell).

4. SMTP (Simple Mail Transfer Protocol)

- ¿Qué es SMTP?

SMTP es el protocolo estándar para el envío de correo electrónico a través de internet. Es el "cartero" de la red que se encarga de entregar los mensajes de un servidor a otro.

- ¿Por qué se necesita SMTP?

Se necesita SMTP para que los correos electrónicos puedan ser enviados de un remitente a un destinatario, incluso si están en diferentes servidores de correo o en diferentes partes del mundo. Proporciona un conjunto de reglas para la comunicación entre servidores de correo, asegurando que los mensajes se enruten y entreguen correctamente.

- ¿Cómo funciona SMTP?

SMTP utiliza TCP, típicamente en el puerto 25 (para el envío de servidor a servidor) o 587 (para el envío de cliente a servidor con autenticación, SMTP seguro).

1. Cuando se envía un correo electrónico, el cliente de correo (o el servidor de correo saliente del usuario) se conecta al servidor SMTP del destinatario (o a un servidor SMTP de retransmisión).
2. Se establece una sesión SMTP. El cliente y el servidor intercambian comandos y respuestas.
3. El cliente envía la información del remitente, el destinatario y el contenido del mensaje.
4. El servidor SMTP del remitente intenta entregar el correo al servidor SMTP del destinatario. Si el destinatario está en el mismo dominio, el servidor de correo del destinatario simplemente lo guarda. Si está en otro dominio, el servidor remitente consulta el DNS para encontrar el servidor de correo del destinatario y luego establece una conexión SMTP con él para entregar el mensaje.

- **Ejemplo de aplicación:** Cuando se envía un correo electrónico desde Gmail a una dirección de Outlook, el servidor SMTP de Gmail se comunica con el servidor SMTP de Outlook para entregar el mensaje. El cliente de correo electrónico (ej. Outlook, Thunderbird) utiliza SMTP para enviar el correo a su propio servidor de correo saliente.

5. POP3 (Post Office Protocol version 3)

- ¿Qué es POP3?

POP3 es un protocolo de aplicación utilizado por los clientes de correo electrónico para recuperar mensajes de correo electrónico de un servidor de correo. Su principal característica es que descarga los correos al dispositivo local del usuario y, por defecto, los elimina del servidor.

- ¿Por qué se necesita POP3?

Se necesita POP3 para permitir a los usuarios acceder y leer sus correos electrónicos desde un servidor central. Es útil para usuarios que desean mantener una copia local de sus correos y no necesitan sincronización entre múltiples dispositivos, o para quienes tienen conectividad intermitente y prefieren trabajar sin conexión.

- ¿Cómo funciona POP3?

POP3 opera sobre TCP, típicamente en el puerto 110 (para POP3 estándar) o 995 (para POP3 seguro, POP3S).

1. El cliente de correo electrónico establece una conexión TCP con el servidor POP3.
2. El cliente se autentica con un nombre de usuario y contraseña.
3. Una vez autenticado, el cliente puede:
 - **Listar** los correos disponibles en el servidor.
 - **Recuperar (descargar)** los correos uno por uno.

- **Eliminar** correos del servidor (opcional, pero es el comportamiento por defecto).
- 4. Una vez que los correos han sido descargados, por defecto se eliminan del servidor. Esto significa que si se accede a la cuenta desde otro dispositivo, esos correos ya no estarán disponibles en el servidor.
- **Ejemplo de aplicación:** Un usuario configura su cliente de correo en su computadora de escritorio para usar POP3. Cada vez que el cliente se conecta, descarga todos los nuevos correos a la computadora. Si el usuario luego intenta acceder a su correo desde un teléfono, los correos que ya fueron descargados a la PC no estarán en el servidor (a menos que el cliente POP3 haya sido configurado para dejar una copia en el servidor).
- **Diferencia con IMAP (Internet Message Access Protocol):** Es importante mencionar que IMAP es otra alternativa para recibir correo, y a menudo se prefiere sobre POP3. IMAP permite que los mensajes permanezcan en el servidor y sean sincronizados entre múltiples dispositivos, lo que lo hace más adecuado para usuarios que acceden a su correo desde varias ubicaciones o dispositivos. POP3 es más un "descargar y borrar" mientras que IMAP es un "sincronizar".

6. UDP (User Datagram Protocol)

- ¿Qué es UDP?

UDP es un protocolo de la capa de transporte del modelo TCP/IP. A diferencia de TCP, UDP es un protocolo no orientado a conexión, lo que significa que no establece una conexión formal antes de enviar datos y no garantiza la entrega, el orden o la duplicación de los paquetes.

- ¿Por qué se necesita UDP?

Se necesita UDP en situaciones donde la velocidad es más crítica que la fiabilidad absoluta, o donde la aplicación maneja su propia gestión de errores. Aunque UDP no garantiza la entrega, ofrece un menor overhead (sobrecarga) y una mayor velocidad. Esto lo hace ideal para aplicaciones en tiempo real o que pueden tolerar alguna pérdida de datos.

- ¿Cómo funciona UDP?

UDP opera de forma sencilla:

- Una aplicación envía datos a la capa de transporte, indicando el puerto de destino.
 - UDP simplemente encapsula los datos en un "datagrama UDP", le añade un encabezado simple (que incluye los puertos de origen y destino, y una suma de verificación opcional para detectar errores), y lo pasa a la capa de red (IP) para su envío.
 - No hay "apretón de manos" inicial (handshake), no hay acuses de recibo, no hay retransmisiones de paquetes perdidos y no hay control de flujo o congestión. Si un datagrama UDP se pierde en el camino, UDP no hará nada para recuperarlo. La responsabilidad de gestionar la fiabilidad, si es necesaria, recae en la aplicación.
 - **Ejemplo de aplicación:**
 - **Streaming de video y audio:** Si se pierde un paquete en una transmisión en vivo, es mejor que el video o audio continúe con una pequeña interrupción que esperar a que el paquete se retransmita, lo que causaría un retraso inaceptable.
 - **Juegos en línea:** La latencia es crítica. Una pequeña pérdida de datos es preferible a un retraso que afecte la jugabilidad.

- **DNS (Domain Name System):** Las consultas DNS son generalmente pequeñas y se realizan con frecuencia. UDP es eficiente para este tipo de transacciones rápidas de solicitud-respuesta. Si una consulta se pierde, el cliente simplemente reintenta.
- **SNMP (Simple Network Management Protocol):** Utilizado para monitorear dispositivos de red, donde los mensajes pueden ser pequeños y la pérdida ocasional de un mensaje de estado no es crítica.

Diferencias Fundamentales entre TCP y UDP

La suite TCP/IP es un conjunto de protocolos que trabajan juntos. Dentro de la capa de transporte, los dos protocolos principales son TCP y UDP. Aunque ambos se encargan de la comunicación entre aplicaciones, lo hacen de maneras fundamentalmente diferentes, lo que los hace adecuados para distintos tipos de aplicaciones.

Característica	TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Orientación a Conexión	Orientado a conexión. Establece una conexión antes de la comunicación (handshake de tres vías).	No orientado a conexión. No establece una conexión previa; los datos se envían como datagramas independientes.
Fiabilidad	Confiable. Garantiza la entrega de datos, el orden correcto y la ausencia de duplicados.	No confiable. No garantiza la entrega, el orden o la ausencia de duplicados.
Control de Errores	Incluye mecanismos de detección y corrección de errores (retransmisiones, checksums).	Solo incluye un checksum opcional para detección de errores. No hay corrección.
Control de Flujo	Implementa control de flujo para evitar que un emisor rápido sature a un receptor lento.	No hay control de flujo.
Control de Congestión	Implementa control de congestión para evitar sobrecargar la red.	No hay control de congestión.

Unidad 2: Protocolos y Direccionamiento IP - Tema 5: Protocolos Complementarios de TCP/IP

Orden de Entrega	Garantizado. Los segmentos llegan al destino en el orden en que fueron enviados.	No garantizado. Los datagramas pueden llegar desordenados o no llegar.
Overhead (Sobrecarga)	Mayor overhead debido a los mecanismos de conexión, acuses de recibo y control de errores.	Menor overhead debido a su simplicidad.
Velocidad	Más lento debido a la sobrecarga y los mecanismos de fiabilidad.	Más rápido debido a la ausencia de mecanismos de fiabilidad.
Tamaño del Encabezado	Mínimo de 20 bytes.	Mínimo de 8 bytes.
Aplicaciones Típicas	Navegación web (HTTP/HTTPS), transferencia de archivos (FTP, SFTP), correo electrónico (SMTP, POP3, IMAP), acceso remoto seguro (SSH).	Streaming de medios (video, audio), juegos en línea, DNS, VoIP, SNMP.
¿Por qué se usan?	Se utiliza cuando la integridad y el orden de los datos son críticos (ej. un archivo de texto no puede tener caracteres faltantes o desordenados).	Se utiliza cuando la velocidad y la baja latencia son primordiales, y la aplicación puede manejar la pérdida de paquetes o retransmisiones por sí misma.
¿Cómo funcionan?	Se establece una "conversación" (conexión) antes de enviar datos. Se "confirma" cada paquete enviado. Es como enviar una carta certificada con acuse de recibo.	Simplemente se "lanza" el paquete a la red. No se sabe si llega o no. Es como enviar una postal sin acuse de recibo.